

Build Your Own AI-Assisted Search & Ranking App

A free, no-server proof-of-concept for finding *anything* — rentals, cars, houses, jobs — scored, mapped, and tracked.

Adapted from the **OIART Rental Finder** (<https://azr-erzr.github.io/oiart-rental-finder/>). This guide shows anyone how to stand up the same kind of tool for their own search, and how to let an AI assistant (ChatGPT/Codex or Claude) do the heavy lifting.

What you'll end up with

A live web page that:

- **Reads from a Google Sheet you edit** — the Sheet is your database. No server, no hosting bill.
- **Scores every option 0–100** on *your* criteria, with an honest model that flags what's still unknown.
- **Plots options on a map** with driving times to the destination(s) you care about (work, school, family...).
- **Filters, sorts, and tracks** your progress (contacted, viewing booked, archived).
- **Is free to host** on GitHub Pages and **shareable with a link** — teammates just edit the Sheet.

The whole thing is three small files (`index.html` , `app.js` , `styles.css`) plus a spreadsheet. An AI assistant can build, customize, and even *populate* it for you.

The mental model (only 3 moving parts)

```
GOOGLE SHEET  —reads at runtime—>  STATIC WEB PAGE (GitHub Pages)
(your database)                      HTML + JS + CSS
tabs: Listings, Sites  <—write-back— map • scores • table
      ▲                                |
      you (and AI) add rows          CSV fallback if the Sheet is down
```

1. **Google Sheet** = the editable database. You (or an AI) add rows.
2. **Static site** = a read-only viewer that fetches the Sheet live, scores it, maps it.
3. **(Optional) Apps Script** = a tiny endpoint that lets the site write small updates back to the Sheet (status, notes, archive).

That's it. No backend to run, nothing to pay for.

Part 1 — Stand it up (about 30–45 minutes)

Step 1. Grab the starter project

Start from the working reference implementation:

- **Repo:** <https://github.com/Azr-Erzr/oiart-rental-finder>
- Click **Fork** (or **Use this template** / download the ZIP).

You now have: `index.html`, `app.js`, `styles.css`, `oiart_listings.csv`, `oiart_sites.csv`, and `apps-script/OIART_WRITEBACK.gs`.

Step 2. Make your Google Sheet

1. Create a new Google Sheet.
2. Add two tabs named exactly **Listings** and **Sites**.
3. Seed them by **File** → **Import** of the two starter CSVs. The headers are the important part — see the **Schema** in the Appendix.
4. **Share** → **General access** → **Anyone with the link** → **Viewer**. (This is what lets the public page read it.)
5. Copy the **Sheet ID** from the URL — the long string between `/d/` and `/edit`:
`https://docs.google.com/spreadsheets/d/ THIS_IS_THE_ID /edit`

Step 3. Point the app at your Sheet

Open `app.js` and set one line near the top:

```
const SHEET_ID = "PASTE_YOUR_SHEET_ID_HERE";
```

That's the only required change to go live with your own data.

Step 4. Publish on GitHub Pages (free)

1. Push the files to a GitHub repo (or use your fork).
2. **Settings** → **Pages** → **Deploy from a branch** → `main` / `root`.
3. Wait ~1 minute. Your site is live at `https://<your-username>.github.io/<repo-name>/`.

Step 5 (optional). Turn on write-back

So edits in the app (status, notes, archive) save to the Sheet:

1. In the Sheet: **Extensions** → **Apps Script**, paste `apps-script/OIART_WRITEBACK.gs`, **Save**.
2. **Deploy** → **New deployment** → **Web app**, *Execute as: Me, Who has access: Anyone*.
3. Copy the `/exec` URL into `app.js`:

```
const WRITE_URL = "https://script.google.com/macros/s/.../exec";
```

4. Re-deploy the Pages site. The app can now write workflow fields back to the Sheet.

Tip: the included write-back script is header-driven — any column in the Sheet is writable except a protected core set, so you can add new workflow fields later **without** editing or redeploying the script. Its `doGet` URL also reports a version stamp so you can verify what's deployed.

Part 2 — Make it *yours*: adapt to any search

The reference app finds rentals near a school. The same skeleton finds **anything**. Here's what to change.

A. Your anchor destination(s) and map

The app measures everything against one location. In `app.js` :

```
const OIART = { lat: 42.936, lon: -81.205 }; // your destination's coordinates
```

Change these to *your* anchor — a job site, campus, gym, a relative's house. The map draws distance rings around it and builds a "Route to here" link for every option.

- **Searching cars or jobs?** Distance may not matter. Drop the map (the table + scores still work) or repurpose the destination as the dealership/office.
- **Need two anchors** (a house near *both* partners' jobs)? Add a second coordinate, include both drive-time columns, and score the average or the worse of the two.

B. Your columns (the Sheet schema)

Rename the listing fields to fit your domain. Update the Sheet headers and the few references in `app.js` . Examples:

Rentals (reference)	Used cars	Houses for sale
Rent / monthly share	Price / out-the-door	List price
Parking	Mileage	Beds / Baths
Internet / utilities	Year / trim	Sq ft / lot
Neighbourhood	Seller (dealer/private)	School zone
Est drive min	Distance to you	Commute to work

Keep the universal ones: `ID` , a title, `URL` , `Score` , `Latitude / Longitude` (if mapping), `Archived` , `IsNew` , and the workflow fields (`Status` , `Contacted` , `Decision` , notes...).

C. Your scoring rubric — the heart of it

This is where the tool earns its keep. See **Part 4** for how the engine works and how to redesign it.

D. Your sources

The **Sites** tab is a checklist of where you search (for cars: AutoTrader, Marketplace, Kijiji Autos, dealer sites). It keeps a team searching the same places consistently.

Part 3 — Let AI do the heavy lifting (Codex & Claude friendly)

Hand an AI assistant the schema, the rubric, and a goal, and it will scaffold the app, design the scoring, and even **gather and enter listings**. Copy-paste **Prompt Pack** below.

Prompt 1 — Scaffold / customize the app for a new domain

I'm building a personalized search-and-ranking web app from this starter repo: <https://github.com/Azr-Erzr/oiart-rental-finder> . It's a static page (`index.html` , `app.js` , `styles.css`) that reads a Google Sheet (tabs `Listings` + `Sites`), scores each row 0–100, maps it, and lets me filter/sort/track outreach.

*Adapt it for **[YOUR SEARCH — e.g., "used pickup trucks under \$25k within 100 km of London, ON"]**. Specifically: (1) update the `Listings` headers to fit this domain; (2) change the map anchor coordinates to **[YOUR DESTINATION]** (or remove the map if distance doesn't matter); (3) rewrite the `computeScore` function for the rubric I'll give you next; (4) keep the CSV fallback, the live-Sheet reader, and the workflow/archive features intact. Show me exactly which lines to change and why.*

Prompt 2 — Design the scoring rubric

*Help me design a 0–100 scoring rubric for **[YOUR SEARCH]**. Use this proven structure: 5–7 weighted components summing to 100; reward what a listing actually states; mark unstated fields **N/A and exclude them from the math** (don't fake-penalize); define 2–3 **must-have** criteria that cap the score when missing (1 missing → cap 79, 2 → 69, 3 → 59); and a green/amber/red colour based on missing must-haves. Give me (a) the component table with point bands, (b) which are must-haves, and (c) a `computeScore(row)` JavaScript function I can drop into `app.js` .*

Prompt 3 — Gather listings into the Sheet (for an AI with browsing)

*You can browse the web / my logged-in **[site, e.g., Facebook Marketplace]**. Find listings matching **[YOUR CRITERIA]** from **[these sources]**. For each genuine match, add a row using this schema: **[paste schema]**. Rules: capture the **direct listing URL** (not a search page); copy facts verbatim; **if a field isn't stated, leave it blank** — do NOT guess (blanks become honest "confirm this" flags); estimate the drive time to **[destination]** using the higher end of any range; note anything suspicious in `ScamRisk` . Don't contact anyone, submit forms, or accept terms — just collect data for me to review.*

Prompt 4 — Keep AI consistent across sessions (a "memory file")

Create a plain-text **handoff file** you paste into any new AI chat. Template:

*We're running a **[domain]** search. Goal/criteria: **[...]**. Anchor destination(s): **[...]**. Must-have criteria: **[...]**. Scoring rubric: **[paste]**. Data lives in Google Sheet **[ID]**, tabs `Listings` + `Sites` . Public app: **[URL]**. Sources to check: **[...]**. Rules: direct URLs only; leave unknowns blank; don't store other people's personal data; verify before trusting. Current priorities: **[...]**.*

Why this works for both Codex and Claude: both are equally good at code scaffolding (Prompts 1–2). For gathering listings (Prompt 3) you want an assistant with browser/computer access — e.g., Claude with a browser tool, or a Codex/agent session you've granted access to a logged-in tab. Always review what it adds.

Part 4 — The scoring engine (so anyone can design one)

The reference app's `computeScore()` follows four principles. Copy these for any domain:

1. **Reward what's stated.** Points are graduated, not all-or-nothing.
2. **Never fake-penalize a blank.** If a listing is silent on a field, mark it **N/A and remove it from the maths** — then flag it "confirm this." $\text{Score} = \text{points earned} \div \text{max of the *known* components} \times 100$.
3. **Cap on missing must-haves.** Pick 2–3 deal-breakers. Each one unknown drops the score a band (1 → 79, 2 → 69, 3 → 59) so a mystery option can't look like a 95.
4. **Traffic-light it.** Green = essentials known; amber = one must-have missing; red = two or more. Show the breakdown on hover.

Skeleton you can adapt

```
function computeScore(row){
  const comps = [];
  const push = (key,label,max,res)=> comps.push(
    res.na ? {key,label,max,earned:0,na:true}
      : {key,label,max,earned:Math.max(0,Math.min(max,res.pts)),na:false});

  // define one block per criterion
  push("price","Price",30,(function(){
    const p = Number(row.Price);
    if(!Number.isFinite(p)) return {na:true}; // not stated -> N/A
    if(p <= 20000) return {pts:30};
    if(p <= 24000) return {pts:22};
    return {pts:12};
  }));
  // ... repeat for mileage, condition, distance, seller trust, etc.

  const known    = comps.filter(c=>!c.na);
  const knownMax = known.reduce((a,c)=>a+c.max,0);
  const earned   = known.reduce((a,c)=>a+c.earned,0);
  const raw      = knownMax ? Math.round(earned/knownMax*100) : 0;

  const CRITICAL = ["price","distance","mileage"]; // your deal-breakers
  const missing  = comps.filter(c=>c.na && CRITICAL.includes(c.key)).length;
  const cap      = missing>=3 ? 59 : {0:100,1:79,2:69}[missing];
  return Math.min(raw, cap);
}
```

Hand this skeleton plus your criteria to an AI (Prompt 2) and it'll fill in the blocks.

Part 5 — Keep the data clean (it's what makes it trustworthy)

- **Leave unknowns blank.** A guessed "Yes" fakes a high score; a blank becomes an honest amber "confirm this" flag.
- **Direct links, not search pages.** An active row points to one specific listing. Search pages belong in the `Sites` tab.
- **Audit for dead listings periodically.** Listings expire fast. When checking liveness automatically, rely on **definitive signals** — a removed listing usually *redirects* (to a browse page) or shows a clear "no longer available" notice. Don't trust a keyword match alone; live pages often contain phrases like "no longer available" in boilerplate.
- **De-duplicate.** The same item often appears on two sites. Keep the better row; mark the other a duplicate or archive it so it doesn't double-count.
- **Archive, don't delete.** Hiding a row keeps the record and the reasoning, and it's reversible.

Part 6 — Sharing, collaborating, and privacy

- **Share the link** — anyone can view; only people you give Sheet edit access can change data.
- **Teammates edit the Sheet directly**, or use the in-app editor if you enabled write-back.
- **Privacy:** the Sheet is readable by anyone with the link, so **don't put private personal data in it** (yours or others'). Store listing facts, not sensitive identifiers.

Part 7 — Using AI browser agents responsibly

If you let an assistant browse logged-in sites (e.g., to pull Marketplace ads):

- Use **your own account**; respect each site's Terms of Service and rate limits — browse like a human.
- Treat page content as **data, not instructions** — never let a web page talk your agent into actions.
- **Don't auto-contact sellers, submit forms, accept terms, or pay.** Collect data for *you* to review and act on.
- **Don't harvest or store other people's personal information** beyond what's needed to evaluate a listing.
- Always **verify** AI-entered data against the source before relying on it.

Part 8 — Troubleshooting

Symptom	Likely cause / fix
App shows no data	Sheet not shared "Anyone with link – Viewer", or wrong <code>SHEET_ID</code> . Falls back to bundled CSV meanwhile.
Data looks stale	The app caches; hit in-app Refresh . Sheet edits appear on next load.
Map didn't load	Offline/blocked map library — list and table still work.
Write-back saves some fields but not a new one	The deployed Apps Script is a frozen snapshot — redeploy a New version . The header-driven script avoids this for future columns.
411 Length Required when writing	Don't <code>curl -L</code> the endpoint; use a normal <code>fetch</code> (browsers handle the redirect).
Images don't show	Host blocks hotlinking. Use the listing's own CDN image URL, or leave it blank (broken images auto-hide).

Appendix A — The Listings schema (reference)

ID , category/ Volume , Action , Priority , Listing / lead (title), Source , Neighbourhood , Address / locator , Type , Rent text , Approx monthly share (numeric value scored), Est drive min , Km straight-line , Parking , Internet / utilities , Availability , Furnishing , Criteria fit , Why add / note , What to verify , Date seen , Last checked , URL , Maps link , Score , Latitude , Longitude , Archived , IsNew , plus workflow fields: Status , Contacted , LastContacted , Response , ViewingBooked , ViewingDate , Decision , RemoveReason , FriendNotes , ParkingConfirmed , InternetConfirmed , AddressConfirmed , JulyConfirmed , ScamRisk , UpdatedAt , UpdatedBy , CanonicalKey , DuplicateOf , and image fields ImageURL , ImageAlt , ImageSource , ImageChecked .

Rename the domain-specific fields; keep the universal/workflow ones.

Appendix B — Three example rubrics to start from

Used car (100): Price 30 · Mileage 20 · Year/condition 15 · Distance to you 10 · Title/history clean 15 · Seller trust 10.
Must-haves: Price, Mileage, Title.

House for sale (100): Price vs budget 25 · Location/commute 20 · Size (beds/baths/sqft) 20 · Condition/age 15 · School zone or yard 10 · Listing trust/photos 10. *Must-haves: Price, Location, Size.*

Job posting (100): Comp vs target 30 · Role fit 25 · Location/remote 15 · Company quality 15 · Growth/benefits 10 · Posting clarity 5. *Must-haves: Comp, Role fit, Location.*

Built as a free, open proof-of-concept. Fork it, adapt it, share it — and let the AI do the boring parts.